Assignment 4 for APML24/25@UU

Irvan Ilahibaks, Iván González, Fabian Linsenmeier, Cas Bouwman
Group 5

Abstract

This study explores the application of reinforcement learning (RL) algorithms to solve the 'robot in a maze' challenge, aiming to develop autonomous navigation capabilities within a maze environment. The project utilizes the Gymnasium framework to implement the environment and facilitates the development of RL agents. The primary focus is on applying two RL algorithms, SARSA and Q-learning, to determine the shortest path through the maze... results and conclusions.

1 Introduction

RL is a powerful machine learning approach that enables agents to make decisions by interacting with an environment, receiving feedback in the form of rewards or penalties. In the context of autonomous navigation, RL algorithms can be applied to teach robots how to navigate complex environments, such as mazes. This assignment focuses on using RL to solve the 'robot in a maze' challenge, which is a practical problem in robotics and artificial intelligence. Using RL algorithms, we can develop intelligent agents capable of learning and adapting to their surroundings, paving the way for more advanced autonomous systems in various fields.

The main problem of this study involves navigating a robot through a maze using reinforcement learning algorithms. The task is to formalize the maze problem into a Markov Decision Process (MDP), implement two different RL algorithms, SARSA and Q-learning, and train the agent to find the optimal path to the exit of the maze. This problem requires a careful formulation of the environment, agent interactions, and the reward system.

The primary objectives of this assignment are as follows.

- Formalize the maze navigation problem as an MDP model.
- Develop RL agents using the SARSA and Q-learning algorithms.
- Train agents to navigate the maze and determine the shortest route to the exit.
- Evaluate and compare the performance of the two RL algorithms in terms of learning efficiency and success in solving the maze.

The paper is organized into the following sections. Section 2 outlines the MDP model used to formalize the maze navigation problem and describes the maze environment. Section 3 presents the RL methods employed, including the Q-learning and SARSA algorithms, followed by the presentation of the results, discussion, and conclusions in sections 4 and 5, respectively.

2 MDP model

In this assignment, the dataset is not your typical collection of numbers or records. Instead, it is a procedurally generated maze environment created using the mazelab and gymnasium libraries. This maze acts as a playground for training RL agents to navigate through complex paths. It is basically a 15x15 grid where each cell represents something specific—like open space, obstacles, the agent's position, or the goal.

The maze itself has 225 possible positions the agent can be in. Each position falls into one of four categories:

- Free space (0): Where the agent can move around freely.
- Obstacles (1): Blocks that prevent movement.
- Agent (2): The player or learning entity that explores the maze.
- Goal (3): The target that the agent needs to reach. All of this is stored as a NumPy array, which makes it super efficient to work with during training.

To better understand the maze, a bunch of visualizations have been generated. These include graphical representations of the maze layout, tracking how the agent moves over time and heatmaps showing which areas it visits the most. These visuals help us figure out how well the learning algorithms are working and spot any challenges the agent might be facing while navigating.

A key part of this environment is the reward system, which helps guide the agent's learning process. The agent gets a positive reward for reaching the goal, which pushes it to find the shortest path. On the other hand, it gets penalties for bumping into obstacles or taking too long, discouraging it from wandering aimlessly. There is also a small penalty for each step to motivate the agent to solve the maze as quickly as possible.

The whole thing works as a Markov Decision Process (MDP), meaning the agent makes decisions based on its current position and moves to the next one based on set rules. It can only move in four directions—up, down, left, or right—following what is called the Von Neumann neighborhood model, which keeps the movement simple and structured.

To keep things interesting, the environment introduces some randomness by procedurally generating different maze layouts. This makes the training process more robust and helps the agent learn to handle various challenges instead of just memorizing one path.

3 Learning Algorithms

For the reinforcement learning, we try out two related control algorithms: Q-Learning and SARSA. They are both control methods which try to select an action for a given state and policy by learning an estimated action value. Both algorithms are briefly explained in this section.

3.1 Q-Learning

The first agent we implement uses Q-Learning to solve the maze. It works by updazting the Q-Values based on temporal difference learning and selects actions based on the ϵ-greedy method. It updates according to the maximum over possible actions at the new state. Since it uses different policies for exploring and updating, it belongs to the class of off-policy methods.

The algorithm repeats for every step:

(i) Choose an action A at the current state according to ϵ-greedy

(ii) Take A and observe the reward R and new state S'

(iii) Update the Q-Value at the current position and the taken action Q(S,A):

$$Q(S,A) = Q(S,A) + \alpha \cdot \Big[R + \gamma \cdot \max_a Q(S',a) - Q(S,A)\Big] \tag{1}$$

(iv) Set the new state to S'

This is repeated until the goal is reached or a maximum number of steps has been taken. Q-Learning contains three main hyperparameters: The discount rate γ, the learning rate α and the exploration rate ϵ for the action selection.

3.2 SARSA

As a second agent, we implement the SARSA algorithm to learn a solution to the maze. It works by updating the Q-Values based on temporal difference learning. It selects **and updates** actions based

on the ϵ-greedy method. SARSA uses on-policy training, i.e. it updates based on data generated by the target policy.

The SARSA-algorithm resembles Q-Learning with some important tweaks. The update function for SARSA reads as follows:

(i) Take the action A determined in the step before

(ii) For the new state S', choose an action A' based on ϵ-greedy

(iii) Update the Q-Value according the action A' you just determined:

$$Q(S, A) = Q(S, A) + \alpha \cdot [R + \gamma \cdot Q(S', A') - Q(S, A)] \tag{2}$$

(iv) Set the state and action to S', A'

SARSA contains three main hyperparameters: The discount rate γ, the learning rate α and the exploration rate ϵ for the action selection.

3.3 Monte Carlo

Monte Carlo is a class of algorithms that tries to approximate the transistion functions by taking a walk through the state action space based on some policy. The difference between the other reinforcement learning algorithms is that Monte Carlo updates does not consider the Q value from a potential next action when updating the Q table. Instead it generates an entire episode and for each state-action pair consider the total reward that would be required to get from that pair to the final pair of the episode. The main Monte Carlo algorithm consists of 2 main parts:
Generate an epsiode:

1. Choose an action A according to a policy π for state S.
2. Take A and observe reward R.
3. Append (S,A,R) to list L

For ease of computation L is often preprocessed in some way. The R values are replaced by the cumulative sum of R weighted using γ when traversing the list backwards.
The second part is updating the Q table. For this there are several different Monte Carlo methods. First let us introduce some useful functions. $N(S, A)$ and $G(S, A)$. Their purpose shall become clear in a moment. However important to note is that at first these functions evaluate to 0 for all state-action pairs, in addition these functions exist across episodes.

The first is Every-visit Monte Carlo. We update Q(S,A) each time the algorithm finds itself in this state-action pair. Every-visit Monte Carlo:

For each (S_t, A_t, R_t) in L we do the following:

1. $N(S_t, A_t) = N(S_t, A_t) + 1$
2. $G(S_t, A_t) = G(S_t, A_t) + R_t$
3. $Q(S_t, A_t) = \frac{G(S_t, A_t)}{N(S_t, A_t)}$

Second we have First-Visit Monte Carlo. Here we update Q(S,A) only the first time we find ourselves in this state-action pair. For this purpose we introduce the following set $visitied = \{\}$. The rest of the function is similar to Every-visit Monte Carlo:

For each (S_t, A_t, R_t) in L we do the following only if $(S_t, A_t) \notin visited$:

1. $N(S_t, A_t) = N(S_t, A_t) + 1$
2. $G(S_t, A_t) = G(S_t, A_t) + R_t$
3. $Q(S_t, A_t) = \frac{G(S_t, A_t)}{N(S_t, A_t)}$
4. $visited = visited \cup (S_t, A_t)$

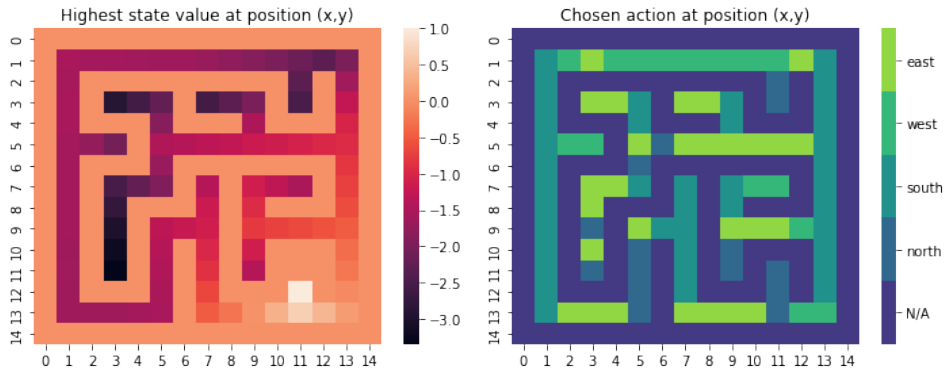

Figure 1: SARSA performance

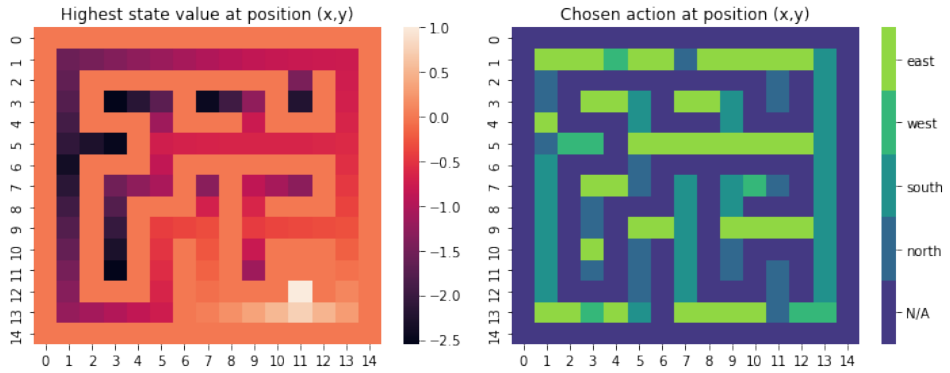

Figure 2: Q learning performance

The final version of the Monte Carlo algorithm can apply to both First-visit and Every-visit Monte Carlo. This version updates the Q table incrementally similar to the SARSA and Q-learning functions derived from the Bellman equations. An additional hyperparameter α is introduced. The example algorithms applies this to Every-Visit Monte Carlo. However it can also apply to first-visit Monte Carlo.

For each (S_t, A_t, R_t) in L we do the following:

1. $N(S_t, A_t) = N(S_t, A_t) + 1$
2. $Q(S_t, A_t) = Q(S_t, A_t) + \frac{\alpha}{N(S_t, A_t)} [R_t - Q(S_t, A_t)]$

Monte Carlo has the following hyper-parameters $epsilon$ for the initial policy, γ the discount factor and α for the incremental learning variant.

4 Results and Conclusion

4.1 Results

Experimentally we found the following optimal hyperparameters for SARSA and the Q-learning: $\gamma = 0.8$,$\epsilon = 0.1$,$\alpha = 0.05$. Thus we are comparing the performance for these algorithms using these hyperparameters.

The left figures show the Q-table at the end of training, while the right figures shows the chosen action at a certain position. SARSA and Q-learning show very similar performance. The main difference between the two is that Q-learning has relatively lower Q-values in the dead ends of the maze. In addition for chosen actions SARSA seems to prefer maintaining the direction it is currently going, while the Q-learning seems more sporadic as it seems to turn more on intersections resulting

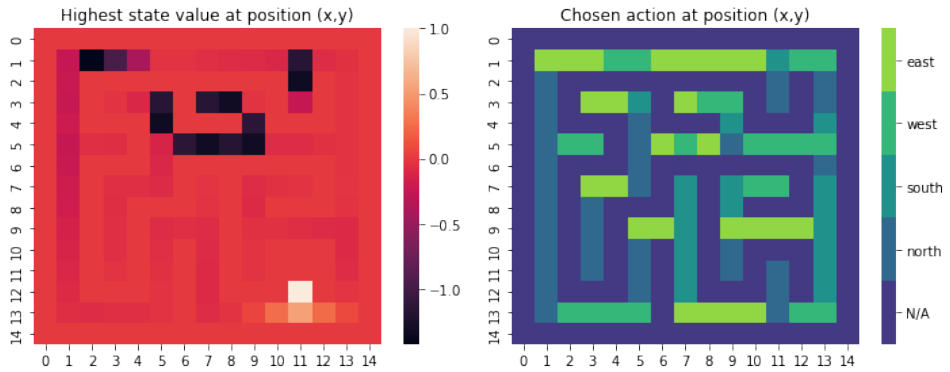

Figure 3: Monte Carlo

in a more chaotic agent. For Monte Carlo we found that Every-visit Monte Carlo performs best at hyperparameters: $\gamma = 0.6$,$\epsilon = 0.3$,$\alpha = 0.7$

The Monte Carlo algorithm has a smaller range of Q-values compared to the previous algorithms. The main difference is that this algorithms does not seem to penalize dead ends. Instead it has seemingly random parts of the maze that are heavily penalized. Essentially creating a blockade in the maze. This might result in the agent getting stuck if it were to start in between these heavily penalized zones.

4.2 Conclusion

From these results we can conclude that in general Q-learning seems to generate shorter routes as it allows for less sidetracking. This is in accordance with the general notion that Q-learning always generates the faster route, while SARSA allows for more exploration thus generally resulting in a longer route. SARSA seems to be able to explore more dead ends.

In comparison to the previous two Monte Carlo performs not as well as it allows the agent to be stuck in some parts of the maze as it does not seem to penalize some part of maze correctly.

5 Discussion

One of the biggest challenges faced during experimentation was figuring out how hyperparameter tuning affects the learning process. Things like the learning rate, discount factor and exploration rate had a huge impact on how well the agent performed. If these values weren't set right, the agent either took forever to learn or got stuck in local patterns without finding the best solution.

Both the SARSA and Q-learning algorithms managed to solve the maze effectively, but their results highlighted the classic trade-off in reinforcement learning—exploration vs exploitation. Should the agent keep exploring new paths or should it stick to the ones it knows work well? Finding the right balance was key to success.

Looking ahead, there are a lot of ways to improve things. Future experiments could try out better exploration strategies, tweak the reward system dynamically or even use function approximation techniques to make the algorithms more scalable and adaptable for even bigger, more complex environments.

6 Appendix

6.1 I. Workload division

Notebook programming

1: Fabian Linsenmeier

2: Fabian Linsenmeier & Cas Bouwman

3: Cas Bouwman

Bonus Tasks:

Report

Abstract: Iván González

Introduction: Iván González

MDP model: Irvan Ilahibaks

RL: Methods: Fabian Linsenmeier

Results and conclusion: Cas Bouwman

Discussion: Irvan Ilahibaks

References